

HW 2B — OWASP Juice Shop

Identification · Secure Login Form · Exploitation Documentation

Student	Siva Sai Deepank Manoj
Email	deeps45@tamu.edu
Public GitHub	https://github.com/deeps45/juice-guard
Course item	HW 2B — OWASP Juice Shop (60 pts)

Rubric map (60 pts): Criterion 1 Identification (20) · Criterion 2 Secure form (20) · Criterion 3 Exploit documentation (20)

Criterion 1 — Identification (20 pts): Juice Shop vulnerabilities & security measures

While exploring OWASP Juice Shop, I focused on login and user-controlled fields. I identified three realistic vulnerabilities and designed security measures for each one.

Vulnerability in Juice Shop	Security measure	How it stops the attack
SQL injection on login Crafted email input can change a concatenated SQL query and skip the password check (classic Juice Shop login challenge).	Use parameterized queries / ORM lookups only. Validate email format before the database call.	User input stays bound as data, so quotes and OR 1=1 cannot change query logic or bypass authentication.
Cross-site scripting (XSS) Search / review fields can reflect or store HTML/JS that runs in another user's browser.	Encode output; use <code>textContent</code> ; reject <code><</code> <code>></code> on input; set <code>Content-Security-Policy</code> .	Scripts are shown as text instead of executed, and CSP blocks inline script even if rendering slips.
Authentication bypass / weak sessions Weak or predictable tokens can let an attacker impersonate another account.	Signed short-lived tokens, server-side authorization on every sensitive route, rate limits, generic login errors.	Stolen or guessed tokens expire quickly; brute force is slowed; attackers cannot enumerate valid emails from error text.

Secure password handling (bcrypt example)

For registration/login design, passwords are never stored in plaintext. I hash with bcrypt (cost 12) and verify with `bcrypt.compare`:

```
const passwordHash = await bcrypt.hash(password, 12);
const ok = await bcrypt.compare(submittedPassword, user.passwordHash);
if (!ok) return res.status(401).json({ error: 'Invalid email or password.' });
```

Criterion 2 — Build a user form with basic security features (20 pts)

I built a Juice Shop-style login form in HTML + JavaScript with an Express backend. Security was part of the design, not an afterthought. The form includes email and password fields, client-side validation, and matching server-side validation.

Security feature in my form	Where	What it does
Client validation	<code>public/app.js</code>	Blocks empty fields; email must contain @; password ≥ 8 characters.
Server validation	<code>server.js</code>	Re-checks email/password; rejects unsafe characters; never trusts the browser alone.
bcrypt password hashing	<code>server.js</code>	Stores only bcrypt hashes (cost 12); login uses <code>bcrypt.compare</code> .

No dynamic SQL	server.js	Users live in an in-memory map keyed by email — input is never concatenated into SQL.
XSS-safe UI + CSP	UI + HTTP headers	Status text via textContent; Content-Security-Policy blocks inline scripts.
Rate limiting	/api/login	Limits repeated login attempts to slow brute-force guessing.

Public GitHub repository (with README): <https://github.com/deeps45/juice-guard>

```
git clone https://github.com/deeps45/juice-guard.git && cd juice-guard
npm install && npm start # http://127.0.0.1:3847
# demo: demo@juice.shop / JuiceShop1!
```

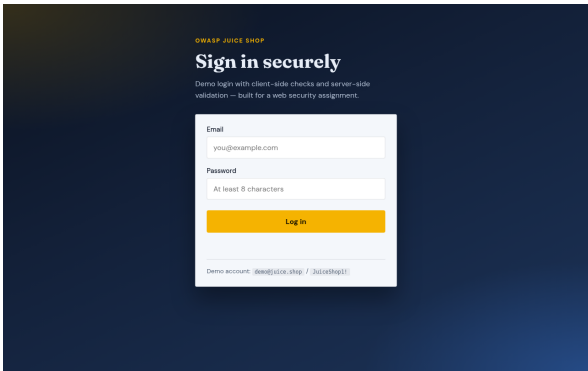


Figure 1. Secure login form UI.

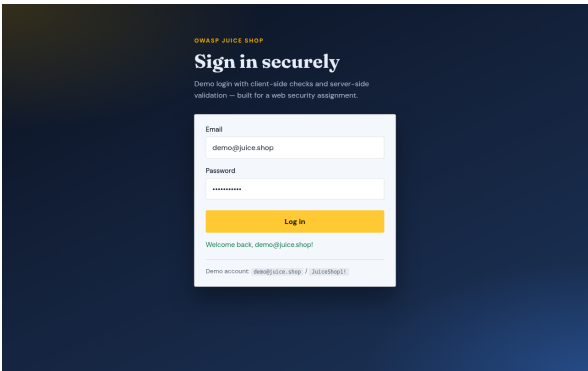


Figure 2. Successful login after checks.

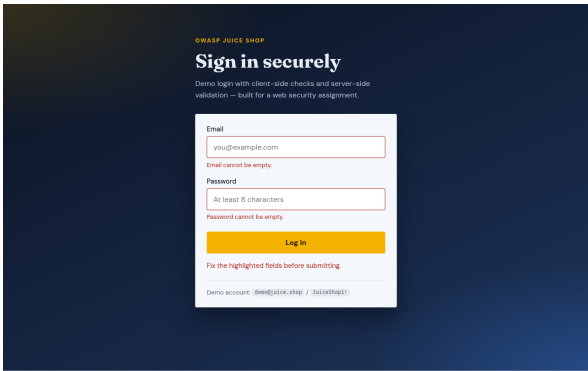


Figure 3. Empty submission blocked.

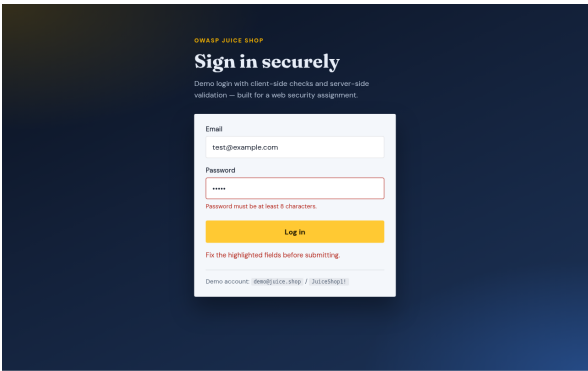


Figure 4. Short password blocked.

Criterion 3 — Identify & exploit weaknesses in my form (20 pts)

I tried to break my own login form the same way Juice Shop is often attacked. Below is organized documentation of each attempt, the exact steps, the result, and a weakness I still found in the design.

Test	Exact input / steps	Outcome	Evidence
XSS	Email: xss@test.com Password: <script>alert(1)</script> Click Log in; watch for an alert.	Blocked. No alert. Server rejected unsafe characters. UI uses textContent.	Figure 5
SQL injection	Email: admin@test.com' OR '1'='1 Password: password1 Submit and check for auth bypass.	Blocked. Invalid email. No SQL concatenation. No credential leak.	Figure 6
Weakness found	Client checks only require @ and password length ≥ 8. The XSS string passes the client and reaches the API — only the server stops it.	Partial weakness: client validation alone is not enough. If server checks were removed, unsafe input would be accepted.	Fix: keep server validation + CSP

Exploit attempt details

XSS steps: open the form → email xss@test.com → password <script>alert(1)</script> → Log in.

Result: no JavaScript alert; server returned an unsafe-character error. Screenshot in Figure 5.

SQL injection steps: reload → email admin@test.com' OR '1'='1 → password password1 → Log in.

Result: rejected as an invalid email; no auth bypass. Screenshot in Figure 6.

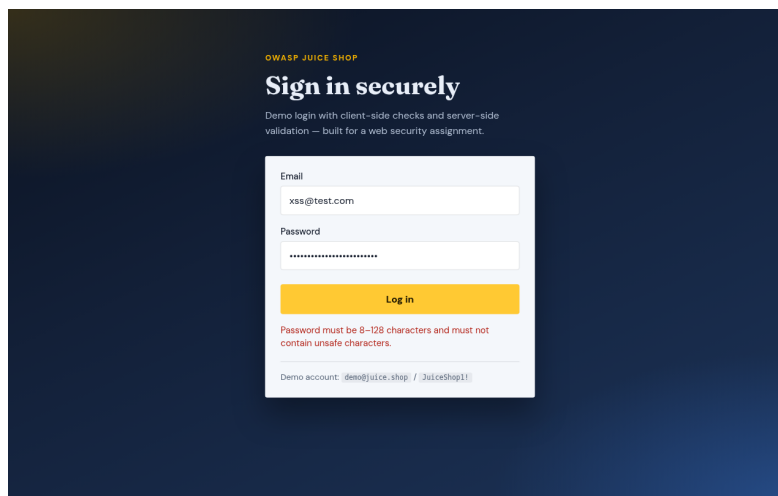


Figure 5. XSS exploit attempt — blocked (no script execution).

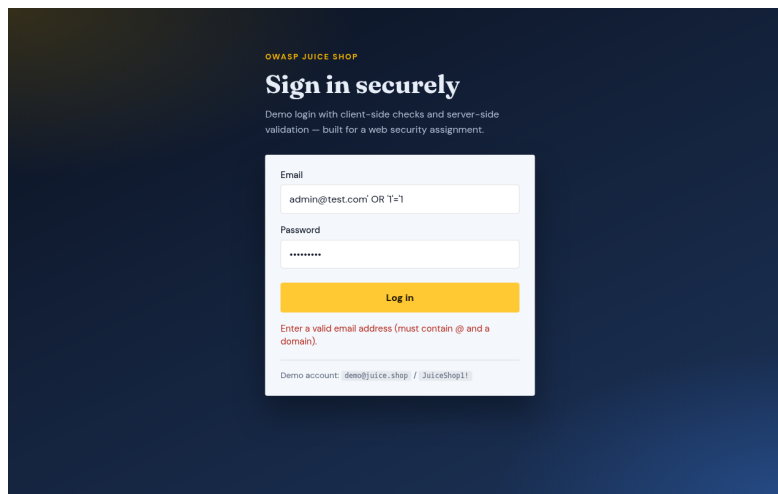


Figure 6. SQL injection exploit attempt — blocked by validation.

Weakness identified + fix

Even though XSS and SQLi did not succeed end-to-end, I found a real design weakness: **the browser-side checks are too weak by themselves**. The client only requires @ and a password length of 8, so the XSS string is sent to the server. If someone later removed server validation, the form would become exploitable. **Fix I applied:** keep strict server-side validation and add a Content-Security-Policy header (`script-src 'self'`) so inline scripts cannot run even if UI code regresses.

Takeaway: failing to fully exploit the finished form is good, but documenting attempts and calling out residual weaknesses (over-reliance on the client) is what shows the security work.

End of HW 2B submission · <https://github.com/deeps45/juice-guard>